
Summarizer
09/02/23
POW!
Code Cadet: Styve Alves

1.Topics:

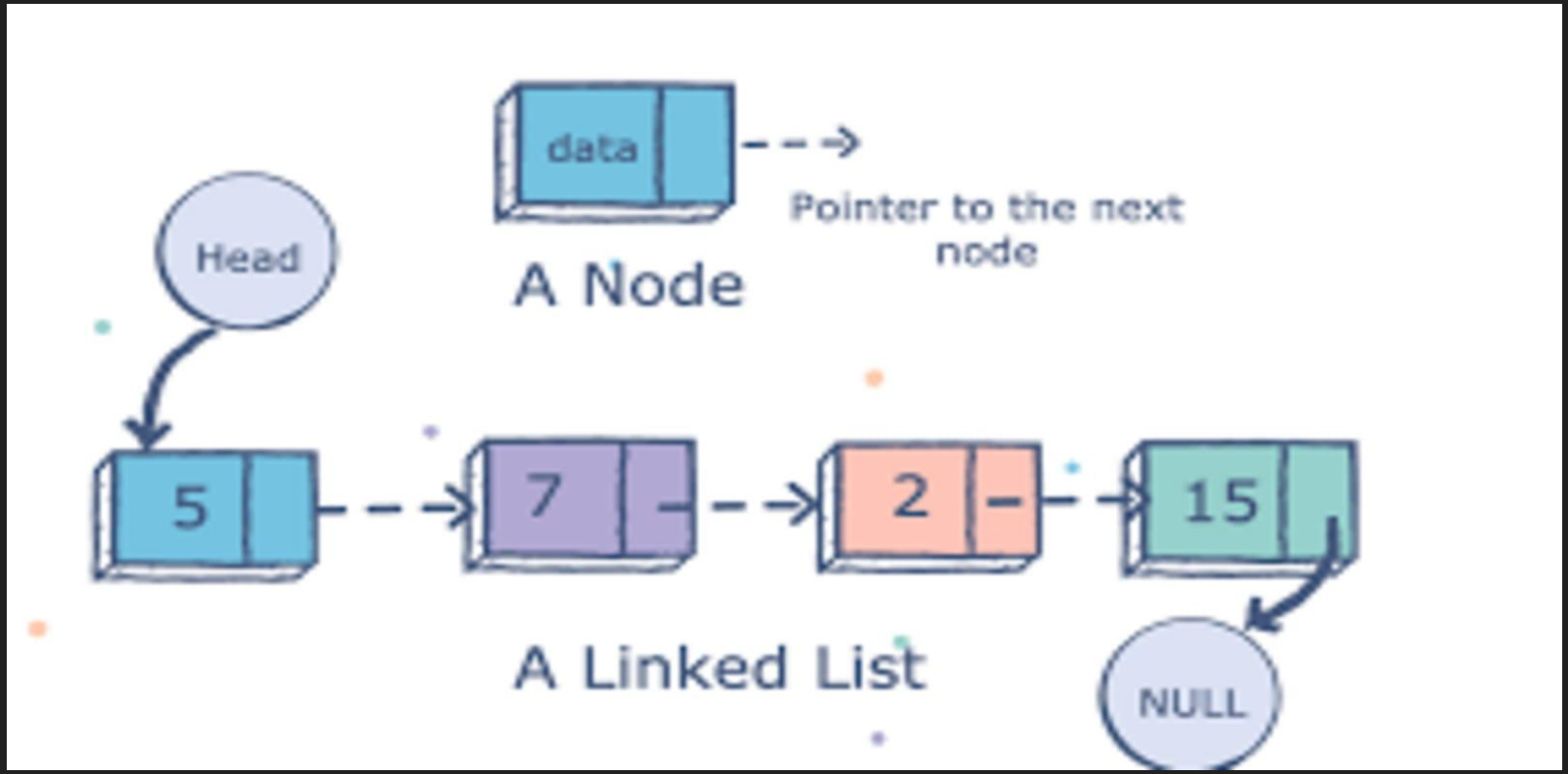
- Container
- Array Vs Linked List
- Linked List
- Exercice

2.Container ?!

3.Array vs Linked List

An array is a collection of elements of a similar data type.	A linked list is a collection of objects known as a node where node consists of two parts, i.e., data and address.
Array elements store in a contiguous memory location.	Linked list elements can be stored anywhere in the memory or randomly stored.
Array works with a static memory. Here static memory means that the memory size is fixed and cannot be changed at the run time.	The Linked list works with dynamic memory. Here, dynamic memory means that the memory size can be changed at the run time according to our requirements.
Array elements are independent of each other.	Linked list elements are dependent on each other. As each node contains the address of the next node so to access the next node, we need to access its previous node.
Array takes more time while performing any operation like insertion, deletion, etc.	Linked list takes less time while performing any operation like insertion, deletion, etc.
Accessing any element in an array is faster as the element in an array can be directly accessed through the index.	Accessing an element in a linked list is slower as it starts traversing from the first element of the linked list.
In the case of an array, memory is allocated at compile-time.	In the case of a linked list, memory is allocated at run time.
Memory utilization is inefficient in the array. For example, if the size of the array is 6, and array consists of 3 elements only then the rest of the space will be unused.	Memory utilization is efficient in the case of a linked list as the memory can be allocated or deallocated at the run time according to our requirement.

4.Linked list



5. Ex. Implement remaining methods until all tests pass:

```java
public int indexOf(POW data) {

        Node iterator = head;
        int count = -1;
        while(iterator.getNext() != null) {

            count++;

            iterator = iterator.getNext();

            if(iterator.getData().equals(data)) {
                return count;
             }

        }
        return -1;


    }
```

```java
public POW get(int index) {


        Node iterator = head;
        int count = 0;

        while (iterator.getNext() != null){
            iterator = iterator.getNext();
            if(count == index){
                return iterator.getData();
            }
            count++;
        }
         return null;
    }
```

5.1 Ex. Implement remaining methods until all tests pass:

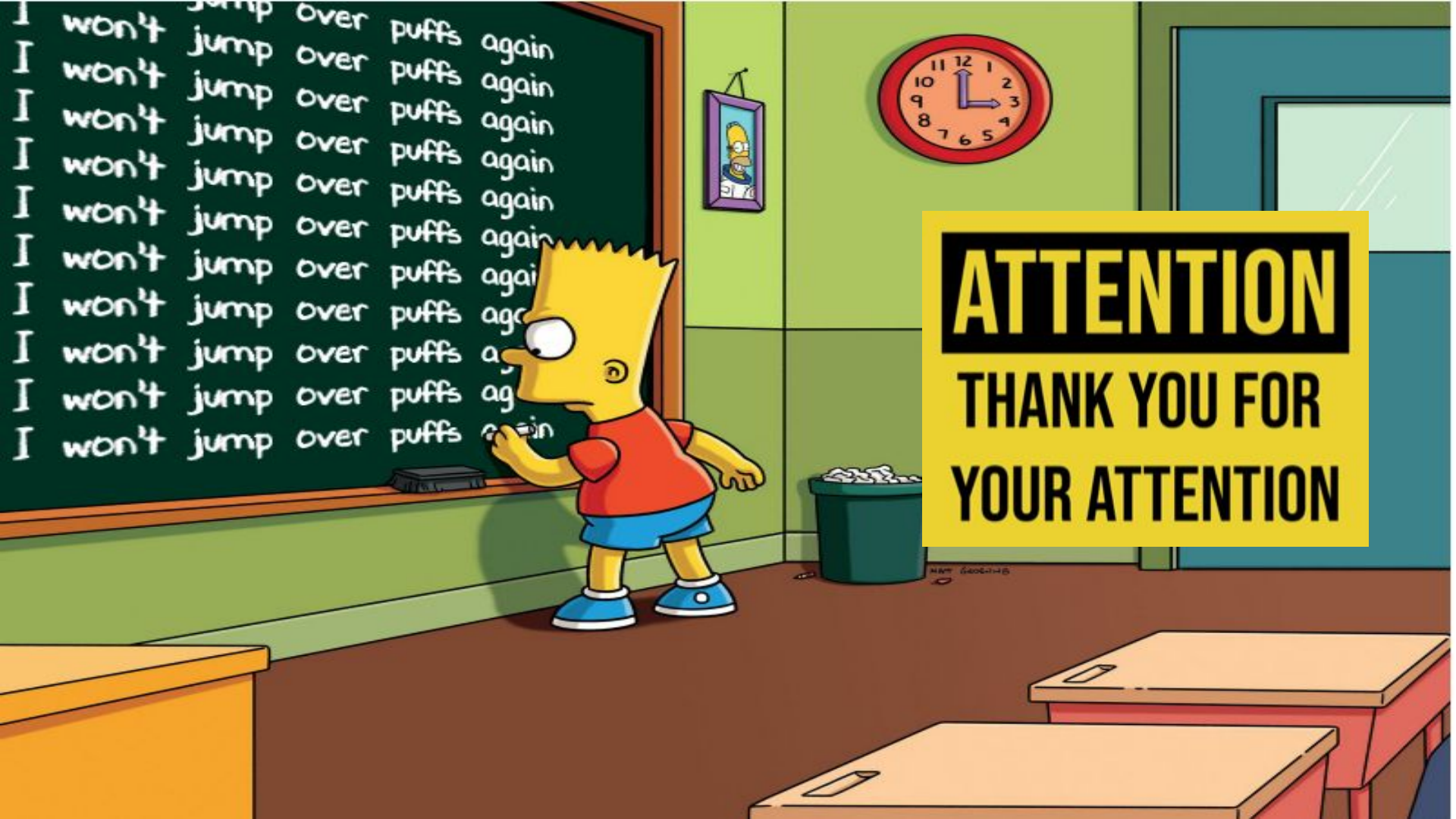
```
public boolean remove(POW data) {

        Node iterator = head.getNext();
        Node iteratorStart = head;

        while (iterator != null) {

            if (iterator.getData().equals(data)) {
                iteratorStart.setNext(iterator.getNext());
                length --;
                return true;
            }
            iterator = iterator.getNext();
            iteratorStart = iteratorStart.getNext();


        }
        return false;
    }
```


I won't jump over puffs again
I won't jump over puffs again
I won't jump over puffs again
I won't jump over puffs again
I won't jump over puffs again
I won't jump over puffs
I won't jump over puffs
I won't jump over puffs
I won't jump over puffs
I won't jump over puffs
ATTENTION
THANK YOU FOR
YOUR ATTENTION